# C++ Concurrency in Action, 2ed.

Brian

April 17, 2026

## Contents

# 1 1. Hello, world of concurrency in C++!

- Concurrency is two or more things happening at the same time
- In programming, this is when two or more things happen in parallel
- Two main reasons to use concurrency:
  1. Seperation of Concerns: Things that pertain to a specific task can be run seperately from the rest of the application
  2. Can run things at the same time, improving performance

# 2 2. Managing Threads

- Every C++ program has at least one thread, which runs `main()`.
- A program can launch other threads that have different functions as entrypoints.
- A thread exits when the function returns, similar to how the program exits when `main()` returns.
- Simple case:

```
void do_some_work();
std::thread my_thread(do_some_work);
```

- `std::thread` works on any callable type, including ctors, where the object gets copied into the thread (invokes copy constructor!)

```
class background_task {
public:
    void operator() () const {
        do_something();
        do_something_else();
    }
};
background_task f;
std::thread my_thread(f);
```

- Prefer uniform initialization when creating a thread, to avoid most vexing parse: if you pass a prvalue to instantiate a thread, this is ambiguous with a function declaration (i.e. `std::thread my_thread(background_task());` is a function declaration),
- To prevent most vexing parse:

```
std::thread my_thread((background_task())); //extra parenthesis
std::thread my_thread{background_task()}; //uniform initialization,
    more idiomatic
```

- To construct a thread, we need a callable; lambda expressions are also commonly used.

```
std::thread my_thread([] {
    do_something();
    do_something_else();
});
```

- Ensure to explicitly state whether to wait for the thread to finish or leave it to run on it's own; if this is not decided before the thread is destroyed, then the program exits (`std::thread` dtor invokes `std::terminate()`)
- Also, it is imperative to ensure that the thread does not access data that it may outlive (important for if you leave the thread to run).
- Same as use-after-free UB, but more tricky to find. Example:

```
struct func {
    int &i;
    func(int& i_): i(i_) {}
    void operator() () {
        for (unsigned j = 0; j < 1000000; ++j) std::cout << i << "\
           n"; //dangling reference
    }
};

void oops() {
    int some_local_state{0};
    func my_func(some_local_state);
    std::thread my_thread(my_func);
    my_thread.detach(); //dont wait for thread to finish
} // local variable some_local_state goes out of scope; thread
   still has reference

int main() {
    oops();
    std::cout << "Here!\n"; //prevent compiler optimizing this away
    return 0;
}
```

- `std::thread::join` will ensure to wait for the associated thread to complete.
- In the above example, replacing `my_thread.detach()` with `my_thread.join()` will make sure that the function exits only after the thread is completed.
- Once `join()` exits, it will clean up all resources associated with the thread, i.e. the `std::thread` object itself is invalid.
- calling `join()` once on a thread will set `joinable()` to be false, i.e. you cannot join the thread anymore.
- Note that a `join()` may be skipped if an exception is thrown after the thread is started but before `join()` is called.
- This can be avoided using a try-catch, however this is very verbose for a pattern that would be used often, and is thus unideal.

```cpp
struct func;
void f() {
    int some_local_state{0};
    func my_func(some_local_state);
    std::thread t(my_func);
    try {
        do_something_in_current_thread();
    } catch {
        t.join();
        throw;
    }
    t.join();
}
```

- It is better to use RAII, and utilize a wrapper type:

```cpp
class thread_guard {
//barebones thread RAII wrapper
    std::thread& t;
public:
    explicit thread_guard(std::thread& t_) : t(t_) {}
    ~thread_guard() {
        if (t.joinable()) t.join();
    }
    thread_guard(thread_guard const&) = delete;
    thread_guard& operator=(thread_guard const&) = delete;
}
```

- Here, `join()` will automatically be called on the thread when it is about to go out of scope.
- However, if the user has explicitly called `join()` already, then it will do nothing.

```cpp
struct func;
void f() {
    int some_local_state{0};
    func my_func(some_local_state);
    std::thread t(my_func);
    thread_guard g(t);
    do_something_in_current_thread();
    // less verbose, and thread_guard is reusable
}
```

- Note that in C++20+, `std::jthread` (joinable thread) was added that basically is a thread with RAII wrapper.
- Calling `detach()` on a thread will leave the thread to run in the background.
- Note that internally, this severes the connection between the actual `std::thread` object and the thread spawned by the os.
- It is thus impossible to communicate with the thread after it gets detached, and you cannot get a `std::thread&` to the detached thread.
- Detached threads are often referred to daemon threads.
- Detached threads are useful for tasks that must run in the background, dont require two way communication, and last for the duration of the program.

- Such tasks include filesystem monitoring, clearing unused entries, optimizing data structures, thread deadlock watchdogs, etc.
- In order to prevent the UB behaviour described above, it is extremely important to ensure that detached threads never access local variables by reference.
- If absolutely needed (for some reason) pass by value, or if it is large then use a `std::shared_ptr`.
- Consider the following example: a word processing application that can support processing multiple documents. This should be ran as a single process, however in order to be able to multiple documents independently each document should have its own thread. Since each dcoument is independent, they should be detached from each other.

```
void edit_document(std::string const& filename) {
    open_document_and_display_gui(filename);
    while (!done_editing()) {
        user_command cmd = get_user_input();
        if (cmd.type == open_new_document) {
            std::string const new_name = get_filename_from_user();
            std::thread t(edit_document,new_name);
            t.detach();
        } else {
            process_user_input(cmd);
        }
    }
}
```

- As seen in the snippet above, passing arguments to the callable function is done by adding more arguments to the `std::thread` ctor.
- By default, this will create a copy of the arguments for the thread, and then are fowarded as rvalues to the function. In the case above, we must ensure to define the signature of f to accept a `std::string const&`, to ensure that it can bind to rvalue references.
- As well, note that the promotion from a `const char*` to a `std::string` only in the context of the new string. Consider:

```
void f(int i, std::string const& s);
void oops(int some_param) {
    char buffer[1024];
    sprintf(buffer, "%i", some_param);
    std::thread t(f, 3, buffer);
    t.detach();
}
```

- In the snippet above:
    1. On the main thread: `oops()` pushes `buffer` to the stack.
    2. `std::thread` ctor is called,
    3. and it sees `char[]` and decays to `char*`.
    4. The thread `t` receives a copy of the address that `buffer` points to.
    5. `t.detach()` is called, and the `oops()` exits.
    6. The stack frame is popped, and `buffer` is freed.

7. The thread now converts `buffer` to a `std::string`, but it contains freed memory.
8. Program crashes due to use-after-free.

- To prevent this, explicitly cast `buffer` to `std::string` when passing to ctor.
- The inverse of this issue can also occur: you want a non-const reference, but the object is copied and passed by value.

```
void update_data_for_widget(widget_id w, widget_data& data);
void oops_again(widget_id w) {
    widget_data data;
    std::thread t(update_data_for_widget, w, data);
    display_status();
    t.join();
    process_widget_data(data);
}
```

- In this case, the program will not compile since `data` is passed by value when it expects a `widget_data&`. Instead of, once again, relying on the implicit conversion, we can explicitly state to pass a reference through `std::thread t(update_data_for_widget, w, std::ref(data))`.